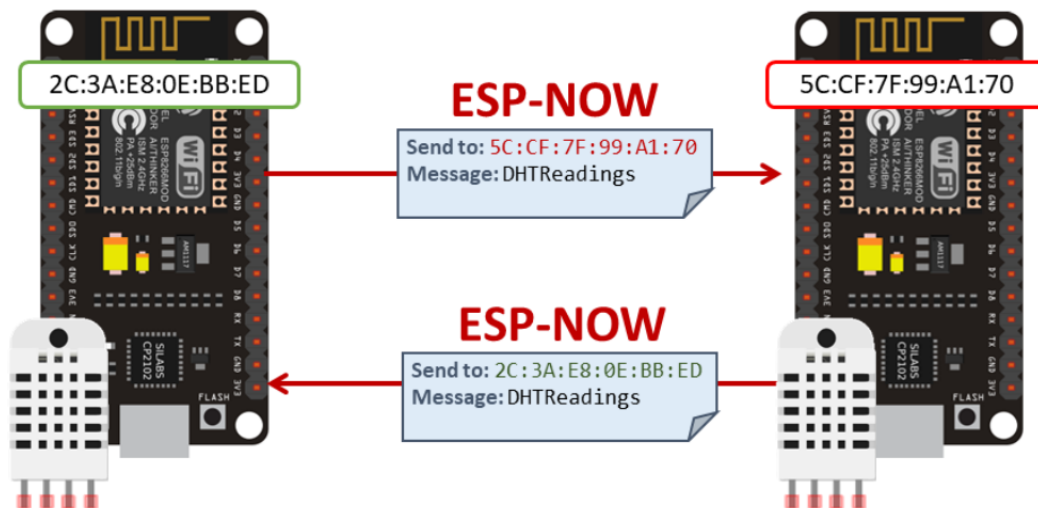


Two-Way Wireless Interlock System

Dual-ESP32 IR-to-Motor Safety Link over ESP-NOW



*Dana Natsheh
Cyber Robot
IoT*

1. Executive Summary

This report documents the design and implementation of a two-way wireless safety interlock built from a pair of ESP32 boards communicating directly over ESP-NOW, with no WiFi router, access point, or cloud service involved. Each board is simultaneously a transmitter and a receiver: it reads its own local IR obstacle sensor and continuously broadcasts that state to its partner board, while at the same time listening for its partner's broadcast and using it to drive its own local DC motor. Neither board is a "server" or a "client" in the traditional sense – both run the identical peer-to-peer logic, differing only in which MAC address is hardcoded as the partner to talk to.

The core safety property of the system is the fail-safe link timeout: if a board stops hearing from its partner for more than two seconds – because the partner lost power, walked out of range, or the RF link was otherwise interrupted – it unconditionally stops its own motor, rather than continuing to run on the last command it happened to receive. This turns a communication failure into a safe, known mechanical state instead of an undefined one.

Four status LEDs per board (transmit heartbeat, motor state, IR/obstacle state, and receive/link state) make the system's internal state observable from the outside without needing a Serial monitor, which proved useful both for the working demo and for isolating faults during bring-up (Section).

2. System Overview & Goal

Two interns, each with one ESP32 board, one IR obstacle sensor, and one DC motor with driver, form a cross-wired safety loop:

```
[Intern A ESP32] --- (IR Sensor) === ESP-NOW ==> (Motor) [Intern B ESP32]
                    tern A ESP32
<--- (Motor) === ESP-NOW === (IR Sensor) [Intern B ESP32]
```

- Intern A's IR sensor state decides whether *Intern B's* motor is allowed to run.
- Intern B's IR sensor state decides whether *Intern A's* motor is allowed to run.
- Both boards run the exact same firmware logic (this report's `main.cpp`); the only per-board difference is the `partnerMAC[6]` array, which must contain the *other* board's WiFi MAC address.

This "your sensor gates my motor" arrangement is precisely what makes it an *interlock*: neither motor can be commanded to run by its own local sensor – only by its remote partner's sensor, over a link that is itself continuously safety-checked (Section).

3. Hardware Architecture & Pin Mapping

Each of the two boards is wired identically. Pin assignments come directly from `config.h`, so there is a single place to retune the wiring without touching the application logic in `main.cpp`.

Peripheral	Signal	ESP32 GPIO	Role
IR obstacle sensor	Digital out	GPIO 34 (IR)	Input-only ADC pin; reads local obstacle state
Motor driver (H-bridge)	IN1	GPIO 26	Direction/enable line 1
Motor driver (H-bridge)	IN2	GPIO 27	Direction/enable line 2 (held LOW – see Section)
Motor driver (H-bridge)	ENA	GPIO 25	PWM speed-enable line, driven once at full duty
Status LED	Transmit heartbeat	GPIO 16 (TransmissionLED)	Toggles every loop iteration whenever a packet is sent
Status LED	Motor state	GPIO 19 (MotorLED)	Mirrors <code>motorShouldRun</code>
Status LED	IR/obstacle state	GPIO 18 (IRLED)	Lit when an obstacle is currently detected
Status LED	Link/receive state	GPIO 17 (RecieveLED)	Lit while the partner link is alive

Table 1: ESP32 pin mapping, identical on both interns' boards.

ESP-NOW channel

Both boards must be configured to the same WiFi channel for ESP-NOW packets to be exchanged; this project fixes it as a single named constant, `ESPNOW_CHANNEL` (channel 1), in `config.h` rather than leaving it as a magic number inside the peer-setup call in `main.cpp`.

4. Schematic and Circuit

Wiring Schematic

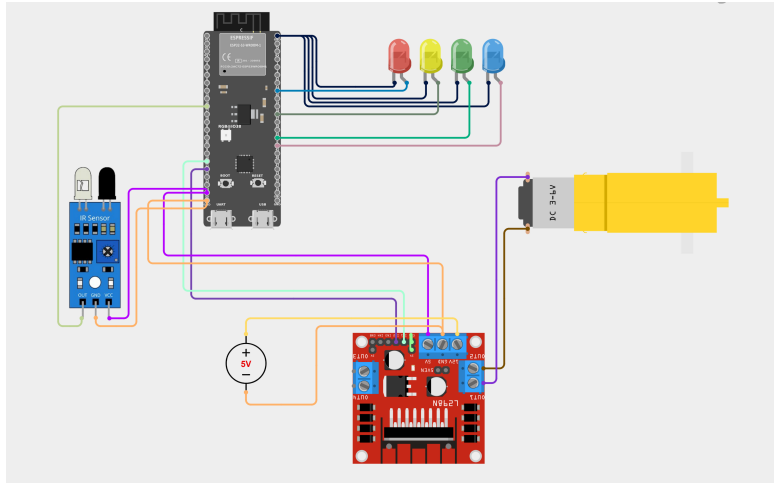


Figure 1: Dual-board wiring schematic

Assembled Circuit

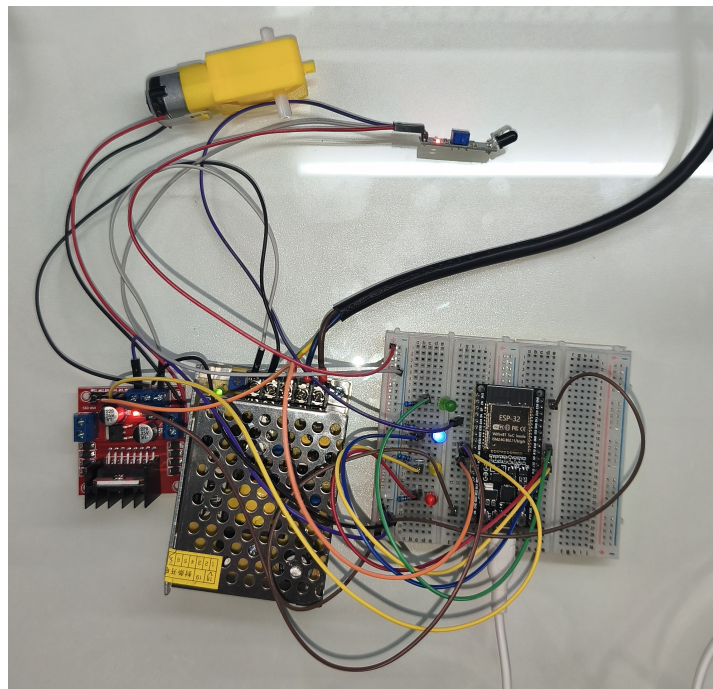


Figure 2: Photo of the two assembled breadboard rigs, positioned so each board's IR sensor faces the area used to trigger the partner's motor

5. Software Architecture

The firmware is split into three small, single-purpose files:

File	Style	Responsibility
config.h	#define constants	Every GPIO pin and the ESP-NOW channel number. No other file contains a raw pin number.
Message.h	Plain C++ struct	The single wire-format message exchanged between the two boards.
main.cpp	Application logic	ESP-NOW init/peer setup, send/receive callbacks, IR debouncing, fail-safe timeout, and motor/LED control.

The wire message

```

1 struct message {
2     uint8_t  senderID;
3     bool     runMotor;
4     uint32_t timestamp;
5 };

```

Listing 1: Message.h – the entire ESP-NOW payload

Keeping this struct in its own header means both boards compile against exactly the same binary layout, and `memcpy`-ing it in and out of the ESP-NOW callbacks (Section) is safe because the struct contains only fixed-width, non-pointer fields – there is nothing in it that would not survive being copied byte-for-byte over the air.

Non-blocking main loop

There is no `delay()` anywhere in `loop()` apart from a short, fixed 20 ms pacing delay at the very end of each iteration. The IR debounce timer, the link-timeout check, and the periodic ESP-NOW send are all evaluated every pass against `millis()`-based timestamps rather than being gated behind blocking waits, so the fail-safe check (Section) is re-evaluated on essentially every iteration and can never be starved by a long blocking call elsewhere in the sketch.

6. ESP-NOW Communication Protocol

Peer setup

At boot, each board puts its WiFi radio into station mode without ever joining a network (`WIFI_STA`, no `WiFi.begin()`), which is all ESP-NOW needs – it operates directly over 802.11 frames and does not require an AP or IP layer at all. The partner is registered once, in `setup()`, as a single unencrypted peer on the fixed channel from `config.h`:

```

1 esp_now_peer_info_t peerInfo = {};
2 memcpy(peerInfo.peer_addr, partnerMAC, 6);
3 peerInfo.channel = ESPNOW_CHANNEL;
4 peerInfo.encrypt = false;
5
6 if (esp_now_add_peer(&peerInfo) != ESP_OK) {
7     Serial.println("Failed to add peer");
8     return;

```

```
9 }
```

Listing 2: main.cpp – peer registration (excerpt)

`partnerMAC[6]` is the one line of the sketch that must differ between the two interns’ copies of the firmware – it must hold the *other* board’s station MAC address, obtainable from that board’s own Serial output at boot.

Non-blocking send and receive callbacks

Both callbacks are intentionally minimal, doing only what is safe to do inside an interrupt-adjacent WiFi callback context:

```
1 void OnDataRecv(const uint8_t *mac_addr, const uint8_t *incomingData, int len) {
2     message incoming;
3     memcpy(&incoming, incomingData, sizeof(incoming));
4     arrive_time = millis();
5     receivedRunMotor = incoming.runMotor;
6 }
7
8 void OnDataSent(const uint8_t *mac_addr, esp_now_send_status_t status) {
9 }
```

Listing 3: main.cpp – callbacks

`OnDataRecv` does exactly two things: it stamps `arrive_time` (the input to the fail-safe timeout, Section) and stores the received `runMotor` flag into a global for `loop()` to act on. No motor logic, printing, or further ESP-NOW calls happen inside the callback itself, which keeps it fast and avoids re-entrant ESP-NOW calls from within a WiFi callback.

Broadcast cadence

Every pass through `loop()`, each board sends its own current, debounced IR state to its partner (roughly every 20ms, bounded by the loop’s pacing delay), stamped with a sender ID and a local `millis()` timestamp:

```
1 message outgoing;
2 outgoing.senderID = 0;
3 outgoing.runMotor = stableObstacleState;
4 outgoing.timestamp = millis();
5
6 esp_now_send(partnerMAC, (uint8_t *)&outgoing, sizeof(outgoing));
```

Listing 4: main.cpp – outgoing message (excerpt)

Sending on every loop iteration (rather than only on a state change) is what makes the fail-safe timeout possible at all: the receiving side is not looking for a “stop” message, it is looking for the *absence* of any recent message, so the sender must keep transmitting continuously even while its own IR state is unchanged.

7. IR Sensor Debouncing

Raw digital IR breakout modules commonly chatter for a short window around the exact obstacle-detection threshold. Rather than trusting the instantaneous `digitalRead(IR)` value directly, `main.cpp` runs it through a simple settle-time debounce before it is ever sent over ESP-NOW or used to drive the local IR LED:

```

1 void updateIRSensor() {
2     bool current_reading = digitalRead(IR);
3     if (current_reading != PrevIRReading) {
4         PrevIRReading = current_reading;
5         last_changed = millis();
6     }
7     if (millis() - last_changed > 500) {
8         stableObstacleState = PrevIRReading;
9     }
10 }

```

Listing 5: `main.cpp` – `updateIRSensor()`

Every time the raw reading flips, the 500 ms settle timer restarts; `stableObstacleState` is only updated to match the raw reading once it has held steady for a continuous 500 ms. This means a single noisy edge or a momentary flicker at the detection boundary cannot, by itself, toggle the partner’s motor – the input has to be genuinely stable before it is trusted and broadcast. The trade-off is an up-to 500 ms delay between a real obstacle appearing/clearing and the partner’s motor reacting, which is an acceptable cost for a safety signal that must not chatter.

8. Fail-Safe Link Timeout

The central safety requirement of the project is that losing the wireless link must *always* result in the local motor stopping – never in the motor continuing to run on stale data. This is implemented with a single, unconditional guard evaluated every loop iteration:

```

1 bool linkLost = (millis() - arrive_time > 2000);
2 bool motorShouldRun = (!linkLost) && receivedRunMotor;

```

Listing 6: `main.cpp` – fail-safe evaluation

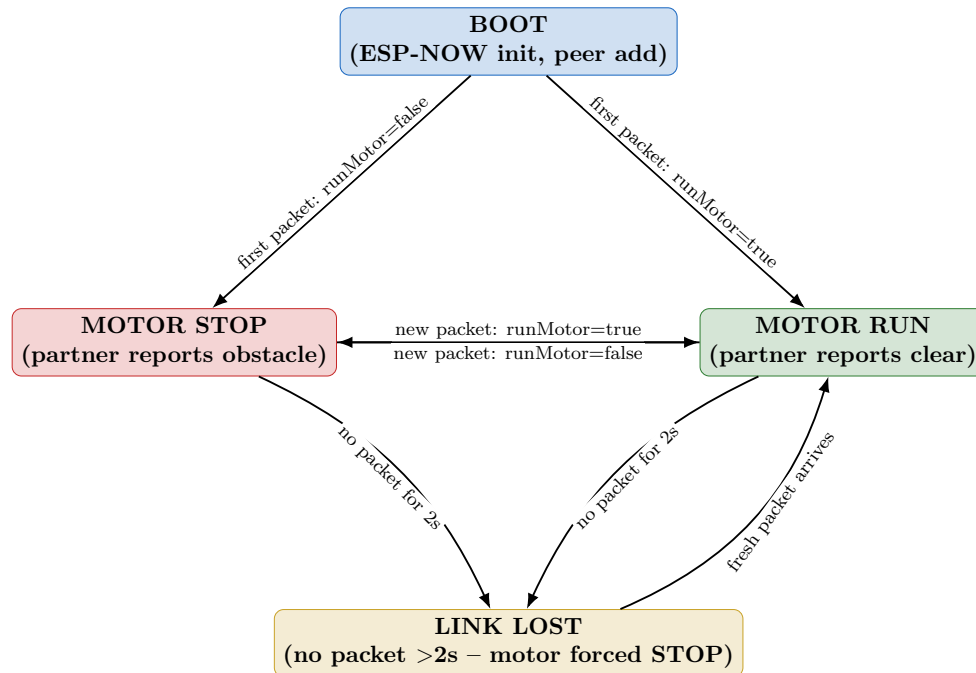
`arrive_time` is only ever updated inside `OnDataRecv`, so it is a direct, tamper-proof record of “the last time a packet actually arrived from the partner.” If more than 2000 ms elapse without a fresh packet – because the partner lost power, was reset, moved out of range, or the RF link was jammed – `linkLost` becomes true and `motorShouldRun` is forced false regardless of whatever `receivedRunMotor` last held. Because this check is a plain boolean AND re-evaluated every pass through the non-blocking loop (Section), the motor cannot be left running on a value that arrived more than two seconds ago.

Why the timeout is measured on arrival, not on content

The timeout deliberately does not care *what* the last received value was – only *when* it arrived. This is an important distinction from, for example, waiting for an explicit “STOP” message: if the partner board loses power mid-broadcast, no further messages of any kind will arrive, including a stop command. Tying the fail-safe to elapsed silence rather than to message content is what makes the interlock safe even against a partner that fails completely, not only against one that is still alive and well-behaved.

9. System State Diagram

The diagram below shows one board's motor-control state machine, driven by the two independent inputs described above: the debounced local IR value received *from the partner*, and the link's liveness.



Reading the diagram

LINK LOST is reachable from either running state and always forces the motor off; leaving it lands directly back in **MOTOR RUN** or **MOTOR STOP** according to whatever the first fresh packet actually contains, rather than through any intermediate "reconnecting" state – there is nothing to reconcile, since `OnDataRecv` always overwrites `receivedRunMotor` and `arrive.time` together, atomically, on every packet.

10. Status LED Indicators

LED	Meaning
Transmit heartbeat (GPIO 16)	Toggles every <code>loop()</code> pass; a steadily blinking LED confirms the board is alive and actively calling <code>esp_now_send()</code> , independent of whether the partner ever answers
Motor state (GPIO 19)	Directly mirrors <code>motorShouldRun</code> – lit exactly when the local motor is being driven
IR/obstacle state (GPIO 18)	Lit when this board’s own debounced IR sensor currently reports an obstacle (<code>!stableObstacleState</code>)
Link/receive state (GPIO 17)	Lit while <code>!linkLost</code> – goes dark the instant the 2-second fail-safe window expires, giving an immediate visual cue of link loss even without Serial

11. Design Notes – Efficient Choices

1. **Silence-based fail-safe instead of a content-based one.** Keying the timeout off elapsed time since the last received packet (Section), rather than off message content, protects against a partner that fails completely – including cases where it never gets the chance to send an explicit stop.
2. **Debounce before broadcast, not after receipt.** The 500 ms settle filter (Section 7) is applied to the *outgoing* value before it is ever sent, so both the broadcasting board’s own IR LED and the partner’s motor react to the same, already-clean signal – there is only one place in the whole two-board system where IR noise could leak through, and it is filtered there.
3. **Single shared struct header.** Because both boards compile the identical `Message.h`, there is no possibility of the two boards silently disagreeing on field order or size – a classic source of hard-to-debug ESP-NOW payload corruption when each side hand-rolls its own copy of a “matching” struct.
4. **Every timing constant centralized in `config.h`.** The ESP-NOW channel and every GPIO pin live in one file, so re-wiring a pin or moving to a different channel is a one-line change rather than a hunt through `main.cpp`.
5. **Non-blocking loop.** With no blocking `delay()` beyond the fixed 20 ms pacing tick, the fail-safe check, the IR debounce timer, and the periodic broadcast are all re-evaluated on essentially every iteration, so the 2-second timeout is actually enforced within roughly 20 ms of its true deadline rather than however long some other blocking call happened to be running.
6. **Visual state without Serial.** All four LEDs together let a person standing in front of either board diagnose “is it alive,” “does it currently see an obstacle,” “is the link up,” and “is the motor

on” at a glance – the same kind of multi-signal, non-ambiguous status feedback that is easy to under-build and then regret once the boards are no longer tethered to a laptop.

12. Known Limitations & Considerations

- **One-directional motor drive.** IN2 is always held LOW; only IN1 toggles. The driver is therefore used purely as an on/off switch for a single direction, which matches the assignment’s run/stop requirement but does not exercise the H-bridge’s reverse capability.
- **No delivery acknowledgement beyond the fail-safe.** `OnDataSent` does not check `status`, so an individual failed transmission is not logged – it is instead implicitly tolerated by the 2-second timeout, which is appropriate here since packets are sent continuously and losing any one of them has no lasting effect as long as the next one arrives within the window.
- **Unencrypted ESP-NOW link.** `peerInfo.encrypt = false` is appropriate for a short-range classroom demo but would need revisiting (encrypted peering, or an application-layer authentication field in `message`) before use anywhere the RF link itself could not be trusted.
- **Hardcoded peer MAC.** `partnerMAC[6]` must be manually edited and reflashed if either board is swapped for a replacement unit, since there is no runtime pairing/discovery step.

13. Results & Verification

Serial verification

With `Serial.begin(115200)` active, each loop iteration prints the current `linkLost`, `receivedRunMotor`, and `motorShouldRun` values, which was used during bring-up to confirm the fail-safe logic transitioned correctly in real time as the partner board was power-cycled.

```
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
linkLost: 0 | receivedRunMotor: 1 | motorShouldRun: 1
```

Figure 3: Serial monitor showing `linkLost` / `receivedRunMotor` / `motorShouldRun` during a live obstacle test

Interlock verification

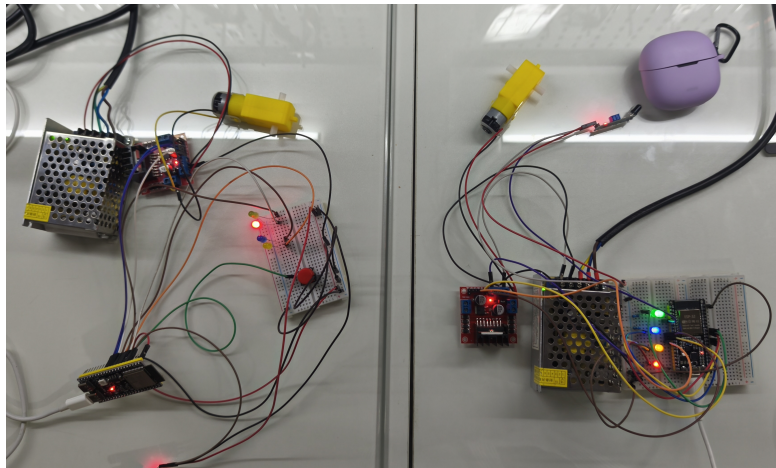


Figure 4: Intern A blocks their IR sensor; Intern B's motor stops within one debounce window

Fail-safe verification

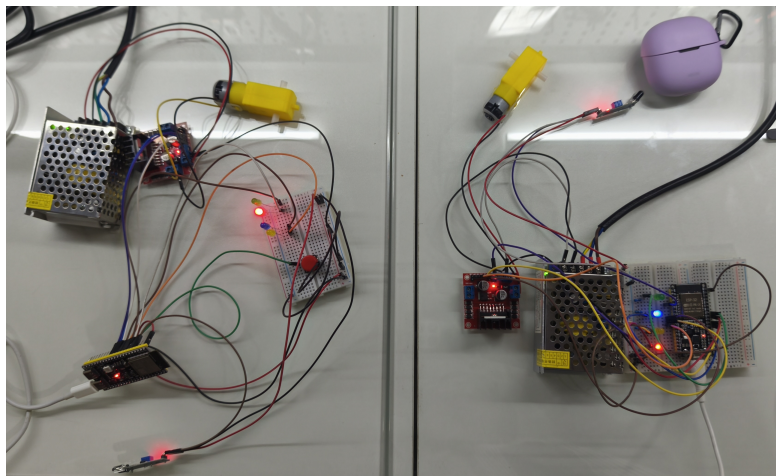


Figure 5: Intern B's board is powered off; Intern A's motor stops within 2 seconds and the Link LED goes dark

14. Bill of Materials

Item	Qty	Notes
ESP32 DevKit development board	2	One per intern; identical firmware, differing only in partnerMAC
IR obstacle sensor module	2	Digital output, one per board, wired to GPIO 34
DC motor + H-bridge driver module	2	Driven via IN1/IN2/ENA per board
Status LEDs (+ current-limiting resistors)	8	4 per board (transmit, motor, IR, link)
Breadboards + jumper wires	2 sets	–
USB cables	2	Programming and power, one per board

16. Future Improvements

- **Bidirectional motor control** using both IN1 and IN2, if the assignment is extended beyond simple run/stop.
- **Send-status checking** in `OnDataSent`, surfaced on an additional LED or Serial line, to distinguish "partner unreachable" from "my own radio failed to transmit."
- **Encrypted ESP-NOW peering** for any deployment outside a fully trusted, physically controlled classroom setting.
- **Runtime MAC pairing** (e.g. a broadcast discovery handshake) to remove the need to hand-edit and reflash **partnerMAC** whenever a board is swapped.